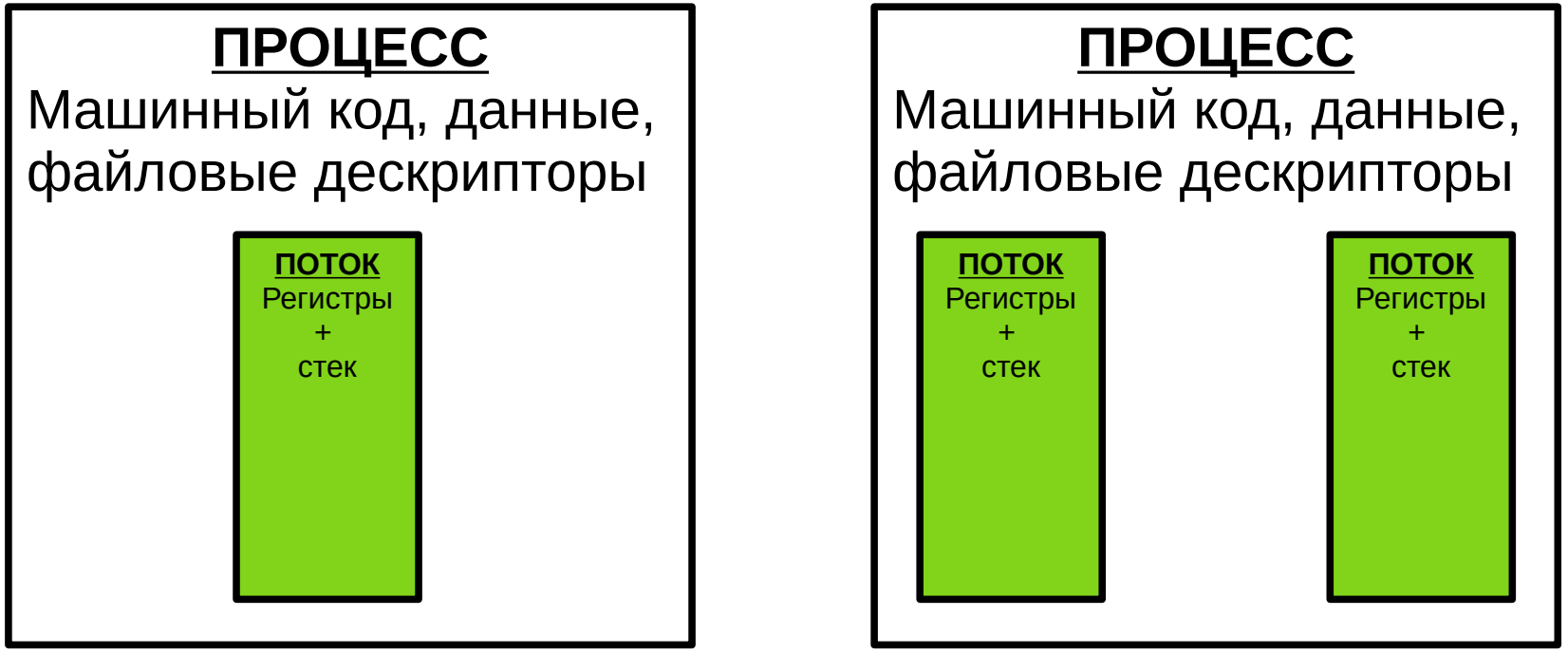
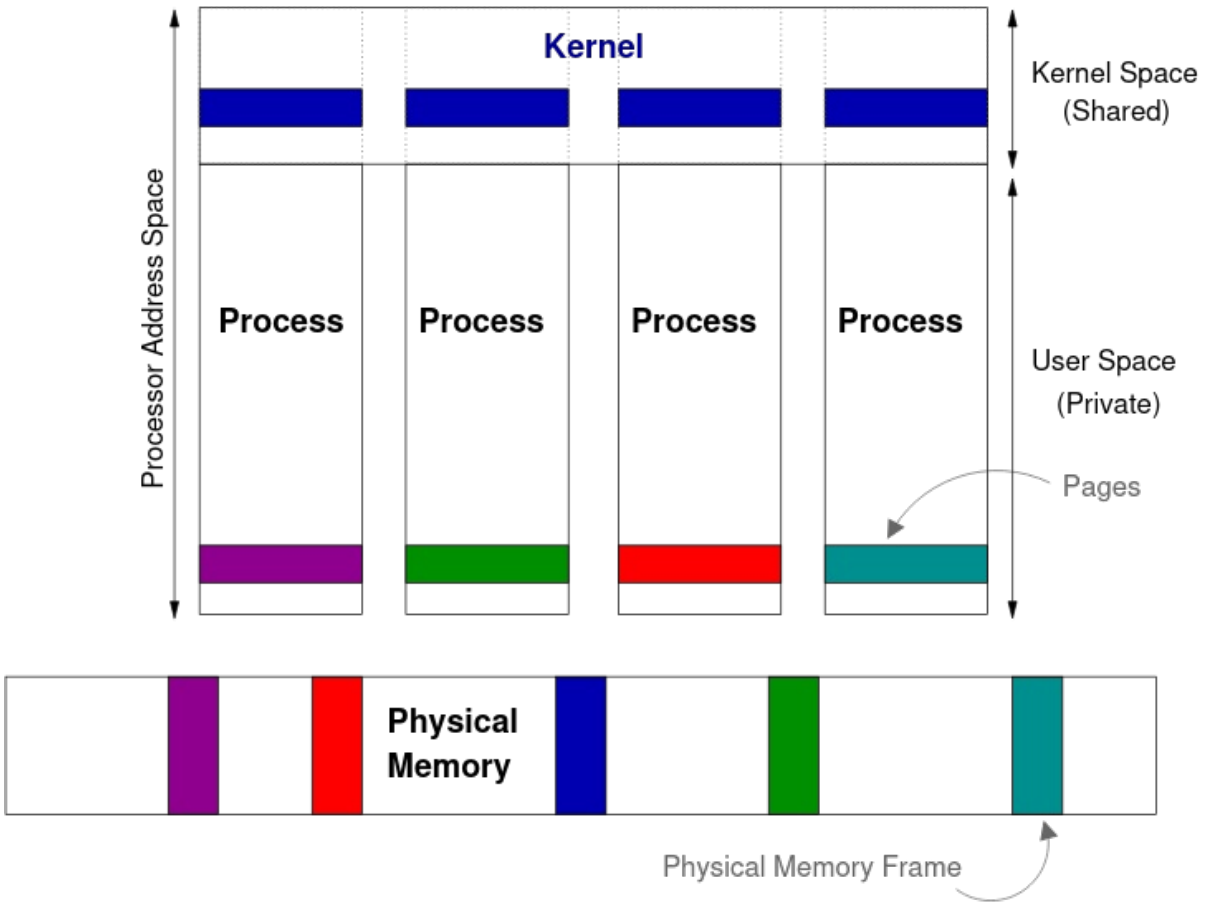
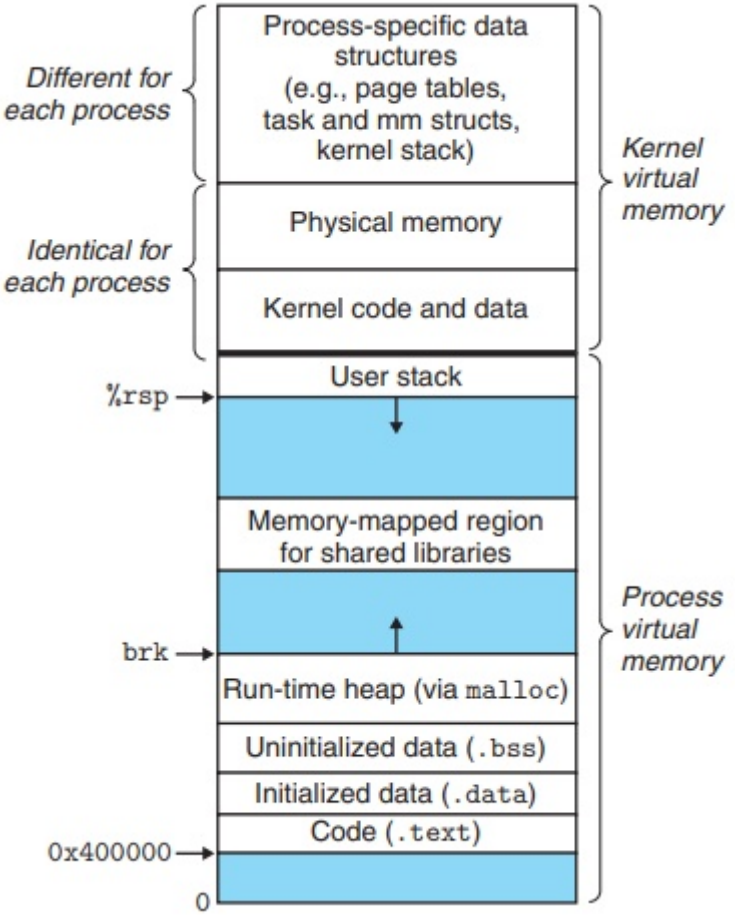
Процессы и потоки.
Пользователи, группы и права доступа.

Процесс vs. Поток



Процесс


Kernel
Kernel Space
(Shared)
Processor Address Space
Process
Process
Process
Process
User Space
(Private)
Pages
Physical
Memory
Physical Memory Frame


Different for each process
Process-specific data structures (e.g., page tables, task and mm structs, kernel stack)
Physical memory
Identical for each process
Kernel code and data
Kernel virtual memory
%rsp
User stack
Memory-mapped region for shared libraries
brk
Run-time heap (via malloc)
Uninitialized data (.bss)
Initialized data (.data)
Code (.text)
Process virtual memory
0x400000
0

procfs

```
$ ls /proc
1        11    124   142   155   170   1828  195  294  37   490  52  69  83   98           ioports      slabinfo
10       110   1242  143   156   171   183   196  3    372  491  53  7   84   99           irq          softirqs
100      111   1244  144   157   172   1839  2    30   373  492  54  70  843  994          kallsyms     stat
1008     112   125   1442  158   1720  184   20   301  378  493  540 71  848  acpi         kcore        swaps
1009     1125  126   145   1584  1724  1841  200  304  38   494  541 72  85   asound       keys         sys
101      113   127   146   159   173   1844  204  31   39   495  55  73  854  bootconfig   key-users    sysrq-trigger
1014     1139  128   1465  16    1738  185   208  32   4    496  550 74  86   buddyinfo    kmsg         sysvipc
1018     114   129   147   160   174   1850  21   328  40   497  56  743 87   bus          kpagecgroup  thread-self
102      1145  13    148   161   1741  186   212  33   41   498  57  75  88   cgroups      kpagecount   timer_list
103      1147  130   149   162   1742  187   213  333  42   499  58  76  89   cmdline      kpageflags   tty
104      115   131   1493  163   175   1873  214  334  43   5    59  768 9    config.gz    latency_stats uptime
1045     116   132   15    1635  176   1874  215  336  44   50   6   77  90   consoles     loadavg      version
1049     117   133   150   1636  1769  1879  22   338  45   500  60  78  91   cpuinfo      locks        vmallocinfo
                                                                              crypto       mdstat       vmstat
                                                                              devices      meminfo      zoneinfo
                                                                              diskstats    misc
                                                                              dma          modules
                                                                              driver       mounts
                                                                              dynamic_debug mtrr
                                                                              execdomains  net
                                                                              fb           pagetypeinfo
                                                                              filesystems  partitions
                                                                              fs           schedstat
                                                                              interrupts   scsi
                                                                              iomem        self
```

```
$ ls /proc/1
arch_status          environ            maps           personality   status
attr                 exe                mem            projid_map    syscall
autogroup            fd                 mountinfo      root          task
auxv                 fdinfo             mounts         sched         timens_offsets
cgroup               gid_map            mountstats     schedstat     timers
clear_refs           io                 net            sessionid     timerslack_ns
cmdline              ksm_merging_pages  ns             setgroups     uid_map
comm                 ksm_stat           numa_maps      smaps         wchan
coredump_filter      latency            oom_adj        smaps_rollup
cpu_resctrl_groups   limits             oom_score      stack
cpuset               loginuid           oom_score_adj  stat
cwd                  map_files          pagemap        statm
$ cat /proc/1/cmdline
/sbin/initsplash
```

procfs self

```
$ cat /proc/self/environ
SHELL=/bin/bashSESSION_MANAGER=local/
elephant:[...]
```

```
$ ls /proc/self
arch_status          environ              maps            patch_state     statm
attr                 exe                  mem             personality     status
autogroup            fd                   mountinfo       projid_map      syscall
auxv                 fdinfo               mounts          root            task
cgroup               gid_map              mountstats      sched           timens_offsets
clear_refs           io                   net             schedstat       timers
cmdline              ksm_merging_pages    ns              sessionid       timerslack_ns
comm                 ksm_stat             numa_maps       setgroups       uid_map
coredump_filter      latency              oom_adj         smaps           wchan
cpu_resctrl_groups   limits               oom_score       smaps_rollup
cpuset               loginuid             oom_score_adj   stack
cwd                  map_files            pagemap         stat
```

```
$ ls -l /proc/self/fd
total 0
lrwx------ 1 avp avp 64 окт  1 09:40 0 -> /dev/pts/2
lrwx------ 1 avp avp 64 окт  1 09:40 1 -> /dev/pts/2
lrwx------ 1 avp avp 64 окт  1 09:40 2 -> /dev/pts/2
lr-x------ 1 avp avp 64 окт  1 09:40 3 -> /proc/60521/fd
```

```
$ cat /proc/self/status | head -7
Name: cat
Umask:      0002
State:      R (running)
Tgid: 61248
Ngid: 0
Pid:  61248
PPid: 59741
```

Поток

- Единица исполнения
- Общие в рамках процесса файловые дескрипторы, код и память.
- Содержит: стек, регистры, состояние

```
$ pstree -t 25602
oosplash─┬─soffice.bin─┬─{PipeIPC}
         │             ├─{dconf worker}
         │             ├─{gdbus}
         │             ├─{gmain}
         │             ├─{pool-spawner}
         │             ├─{soffice.bin}
         │             └─{threaded-ml}
         └─{osl_executeProc}
```

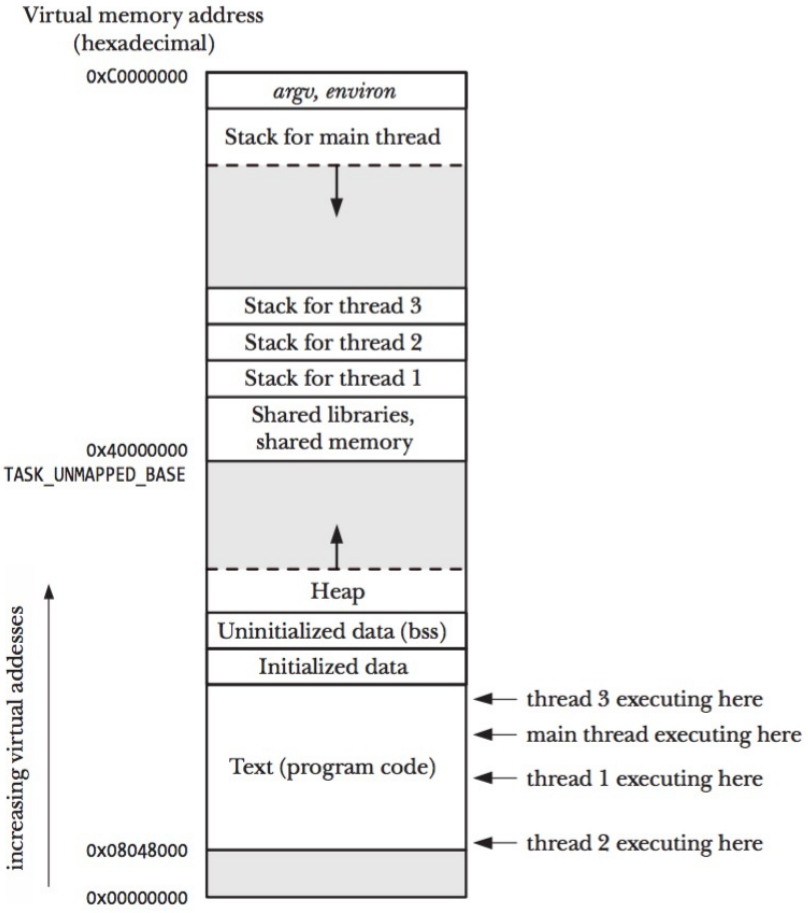
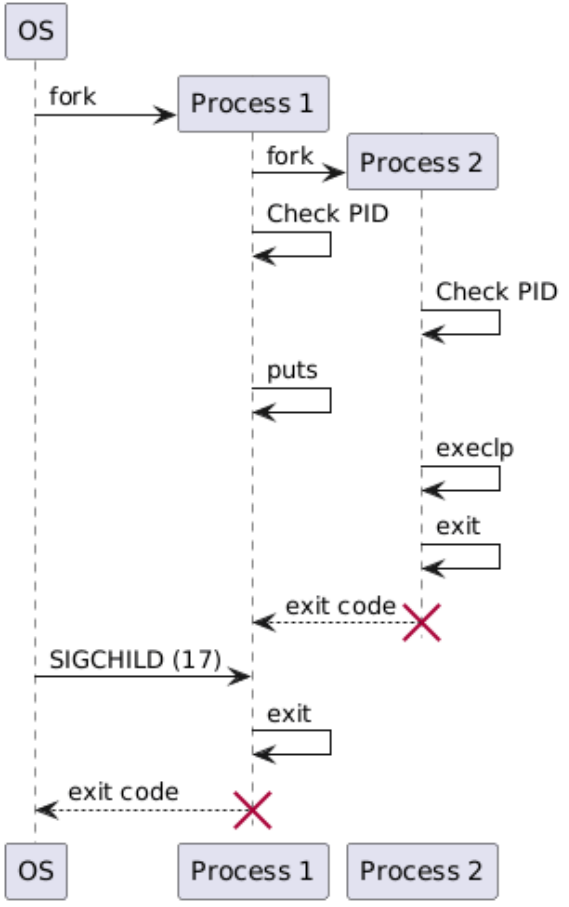


Figure 29-1: Four threads executing in a process (Linux/x86-32)

Создание новых процессов

```
#include <stdio.h>
#include <unistd.h>
int main() {
    int pid = fork();
    if (pid == 0) {
        execlp("/usr/bin/ls", "/usr/bin/ls", NULL);
        puts("Hello from child!");
    } else if (pid > 0) {
        puts("Hello from parent!");
    } else {
        return 1;                   /* ERROR! */
    }
    return 0;
}
```



Статус завершения процесса

```c
#include <unistd.h>
#include <signal.h>
#include <stdlib.h>
#include <sys/wait.h>

void handler(int value) {
    printf("signal: %d\n", value);
}

int main() {
    int i;
    signal(SIGCHLD, handler);
    pid_t pid = fork();
    if (pid > 0) {
        int status;
        waitpid(-1, &status, 0);
        printf("status: %d\n", status);
    } else if (pid == 0) {
        puts("child exit");
        exit(1);
    }

    return 0;
}
```

```
$ gcc main.c
$ ./a.out
child exit
signal: 17
status: 256
```

Системный вызов «clone»

```c
#include <unistd.h>
#include <sched.h>
#include <sys/types.h>
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include <fcntl.h>

int variable, fd;

int do_something() {
    variable = 42;
    close(fd);
    _exit(0);
}

int main(int argc, char *argv[]) {
    void **child_stack;
    char tempch;

    variable = 9;
    fd = open("test.file", O_RDONLY);
    child_stack = (void **) malloc(16384);
    printf("The variable was %d\n", variable);

    clone(do_something, child_stack, CLONE_VM|CLONE_FILES, NULL);
    sleep(1);

    printf("The variable is now %d\n", variable);
    if (read(fd, &tempch, 1) < 1) {
        perror("File Read Error");
        exit(1);
    }
    printf("We could read from the file\n");
    return 0;
}
```

```c
#include <sched.h>

int clone(typeof(int (void *_Nullable)) *fn,
          void *stack,
          int flags,
          void *_Nullable arg, ...
          /* pid_t *_Nullable parent_tid,
             void *_Nullable tls,
             pid_t *_Nullable child_tid */ );
```

Флаги «clone»:
- CLONE_VM – Share the same virtual address space
- CLONE_FS – Share filesystem information
- CLONE_FILES – Share file descriptor table
- CLONE_SIGHAND – Share signal handlers
- CLONE_THREAD – Place the task in the same thread group
- CLONE_SETTLS – Initialize thread-local storage

Аналог «pthread»:

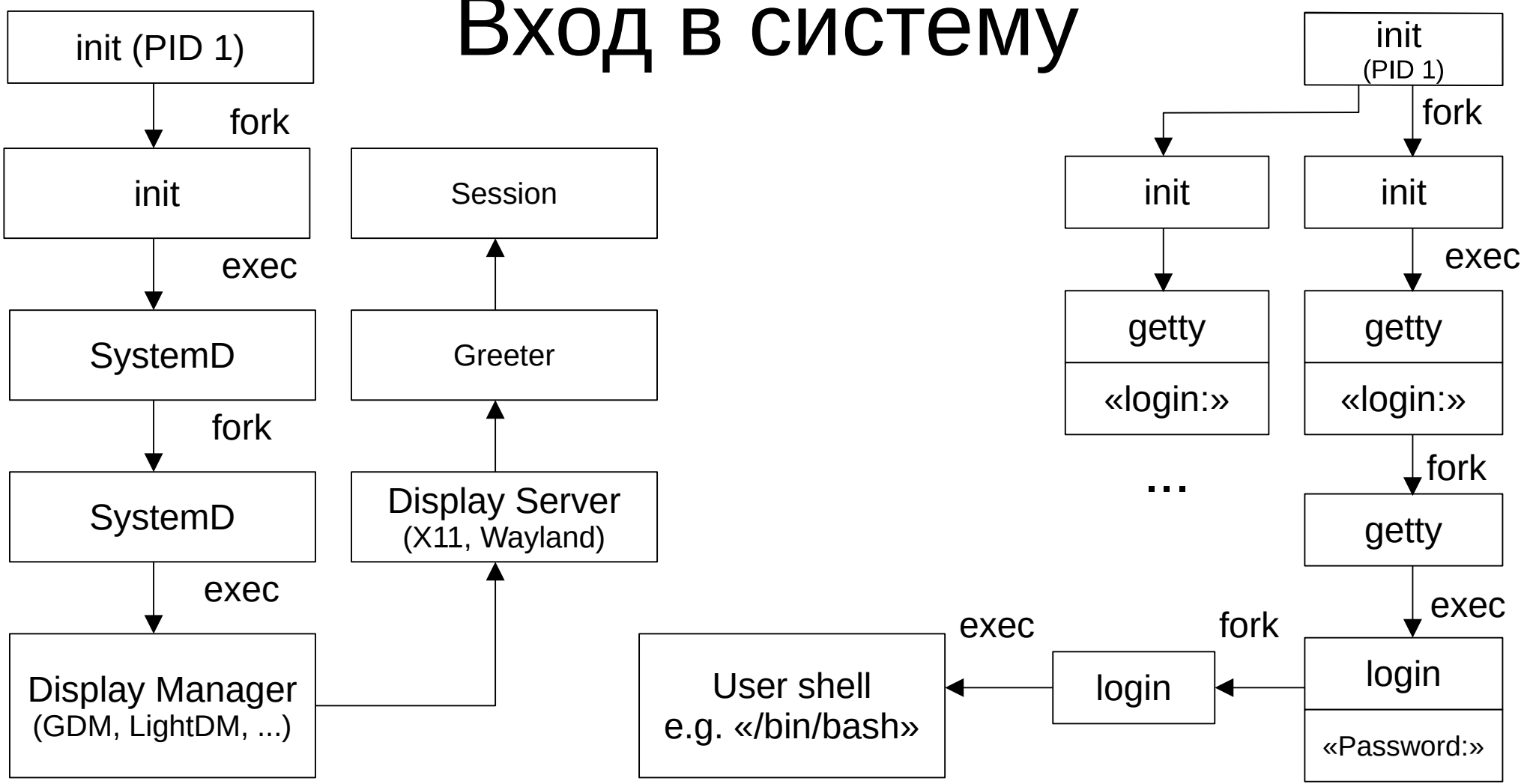
```
clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND, 0);
```

Аналог «fork»:

```
clone(SIGCHLD, 0);
```

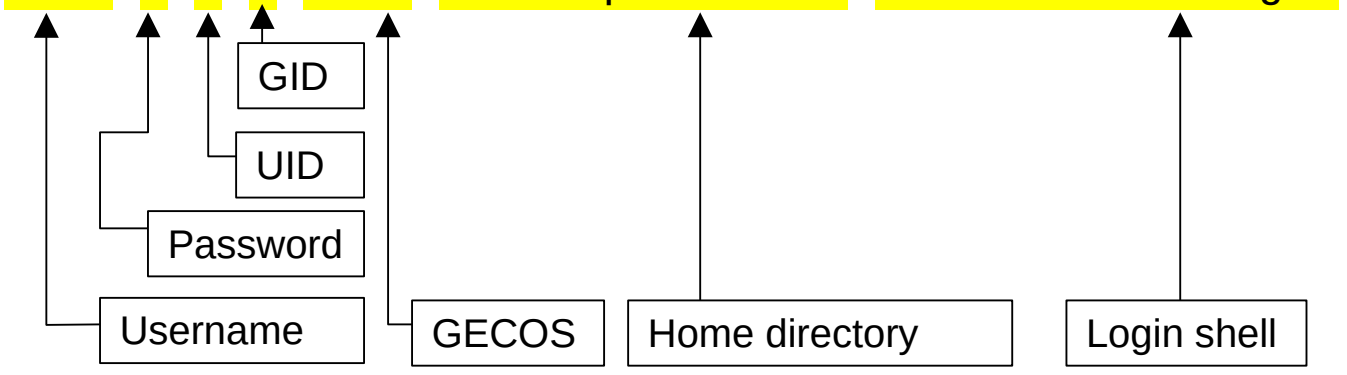
Joey Bernard, «Using the Clone() System Call»:
https://www.linuxjournal.com/article/5211

Вход в систему



Пользователи: /etc/passwd

```
$ cat /etc/passwd | head
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
```



General Electric Comprehensive Operating System (GECOS)
- User's full name (or application name, if the account is for a program)
- Building and room number or contact person
- Office telephone number
- Home telephone number
- Any other contact information (pager number, fax, external e-mail address, etc.)

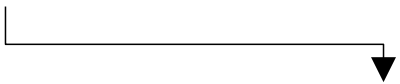
https://en.wikipedia.org/wiki/Gecos_field

Утилита «finger»

```
$ sudo apt-get install finger

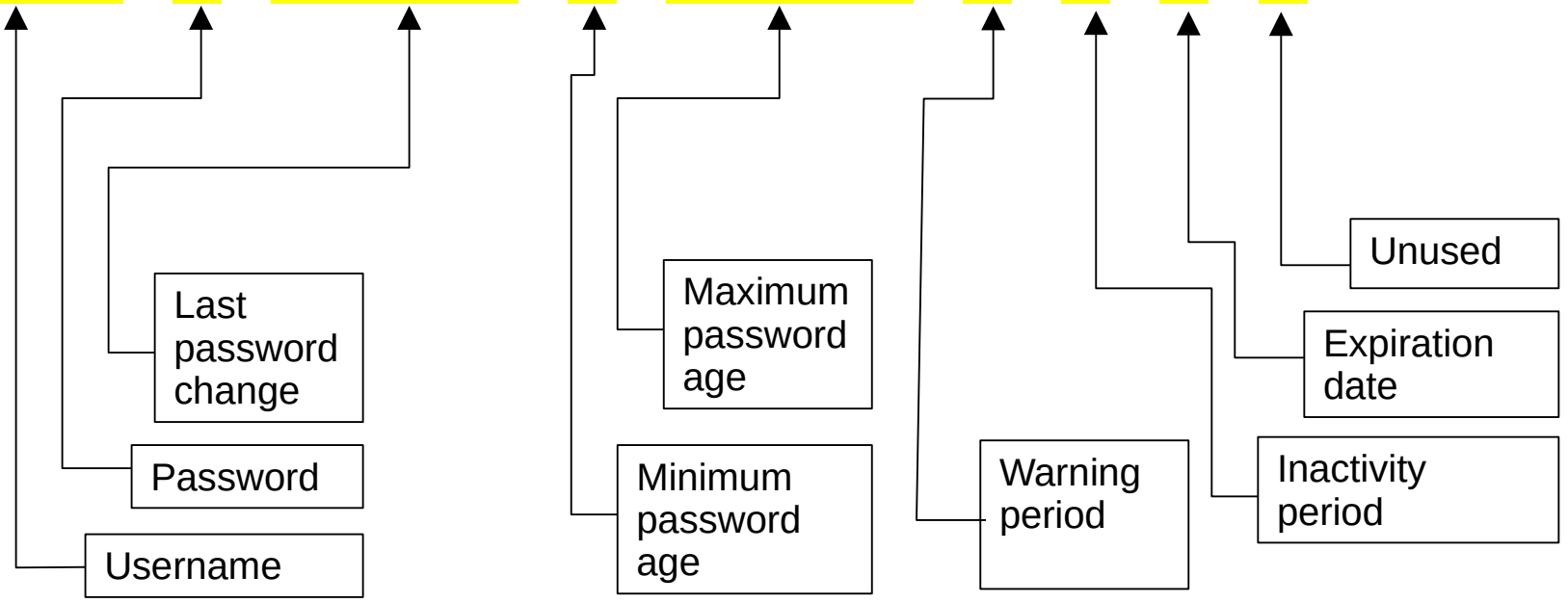
$ grep ^user: /etc/passwd
user:x:1000:1000:Vasiliy Pupkin:/home/user:/bin/bash

$ finger user
Login: user                     Name: Vasiliy Pupkin
Directory: /home/user                 Shell: /bin/bash
Last login Fri Oct  2 09:46 (MSK) on pts/0 from 192.168.122.1
No Mail.
No Plan.
```



Пользователи: /etc/shadow

```
$ cat /etc/shadow | head -2
avp:$6$.n.:17736:0:99999:7:::
root:!:19089:0:99999:7:::
```



The Trusted Computing Base (TCB)

```
$ cat /etc/tcb/root/shadow
root:$y$j9T$YRhJAi4kQLvOF1Y1kVMV90$khdQrVH8NkNIh54Y8A28dpWAedKTJXBVK72FefBEvS/:20512::::::
```

Parameters

Salt

Encrypted password hash

Scheme ID (y = yescrypt)

```
$1$  is MD5
$2a$ is Blowfish
$2y$ is Blowfish
$5$  is SHA-256
$6$  is SHA-512
$y$  is yescrypt
```

```
Modular Crypt Format:
$<id>[$<param>=<value>(,<param>=<value>)*][$<salt>[$<hash>]]

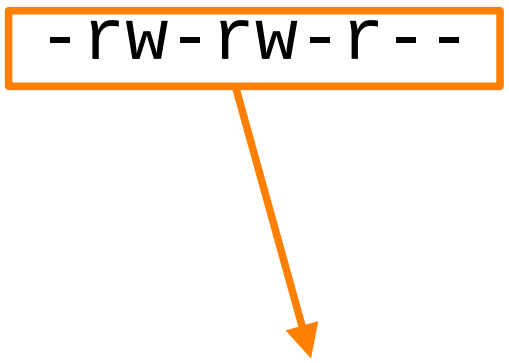
https://en.wikipedia.org/wiki/Crypt_(C)
```



Alexander Peslyak a.k.a. Solar Designer — автор «yescrypt» @ PHDays 2012

Права доступа

```
$ ls -lha main.c
-rwxrwxr-x 1 avp avp 16K сен 30 16:27 a.out
-rw-rw-r-- 1 avp avp 327 сен 30 16:27 main.c
```



```
          owner  group  other     r = read, w = write, x = execute
          r w x  r w x  r - x
binary:   1 1 1  1 1 1  1 0 1
octal:    7      7      5
```

Группы

```
$ cat /etc/group | head
root:x:0:
bin:x:1:root
daemon:x:2:root
sys:x:3:root,bin,adm
adm:x:4:root
tty:x:5:
disk:x:6:root
lp:x:7:
mem:x:8:
kmem:x:9:

$ cat /etc/group | grep ^user:
user:x:1000:
```

```
$ cat /etc/passwd | grep ^user:
user:x:1000:1000::/home/user:/bin/bash
```

```
$ ls -lha /home/user/.bashrc
-rw------- 1 user user 188 июл 26  2021 /home/user/.bashrc
```

Управление правами

Смена владельца:

```
$ chown user:user script.sh
```

Задание прав:

```
$ chmod +x script.sh
$ chmod 700 script.sh
$ chmod u+rwx script.sh
```

Управление процессами в терминале

```
$ emacs &
$ jobs
[1]+  Running                 emacs &
$ fg %1
emacs
^Z
[1]+  Stopped                 emacs

$ bg %1
[1]+ emacs &
$ kill %1
```

Запуск процесса в фоне.

Вывод процесса на передний план.

Процесс остановлен комбинацией клавиш Ctrl+Z

Возобновление работы процесса в фоне.

Сигналы

```
$ kill -L
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL      10) SIGUSR1
11) SIGSEGV      12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT    17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN      22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM    27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS       34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4   39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9   44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14  49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11  54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6   59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1   64) SIGRTMAX

$ pgrep firefox
12345

$ kill 12345
```

Обработка сигналов в Bash

```bash
#!/bin/bash

handler() {
    echo "Ctrl-C is handled"
}

trap handler SIGINT

while true; do
    echo "PID $$ is sleeping..."
    sleep 5
done
```

Terminal 1

```
$ bash main.sh
PID 144978 is sleeping...
PID 144978 is sleeping...
Terminated
```

Terminal 2

```
$ kill 144978
```

Источники изображений

- Linux address-space layout © Ian Wienand — CC BY-SA 3.0: https://bottomupcs.com/ch06s09.html
- Linux process virtual-memory layout (a derivative work) © Randal E. Bryant and David R. O'Hallaron: https://csapp.cs.cmu.edu/3e/ics3/vm/linuxrtimage.pdf
- Multiple threads in a process: virtual address-space figure © Michael Kerrisk: https://michaelkerrisk.com/tlpi/index.html
- Alexander Peslyak / Solar Designer at PHDays 2012 © Ptsecurity — CC BY-SA 4.0: https://commons.wikimedia.org/wiki/File:Solar_designer.jpg